



UNIVERSIDAD POLITÉCNICA DE PACHUCA

INGENIERÍA EN SOFTWARE MANTENIMIENTO DE SOFTWARE

**ACTIVIDAD ALII: REINGENIERÍA DE PROCESOS DE
MANTENIMIENTO (ISO/IEC/IEEE 14764)**

PROYECTO: YOLIZTLI

AUTOR:

ISLAS LOZADA EMMANUEL

CATEDRÁTICO:

ALICIA ORTIZ MONTES

PACHUCA DE SOTO, HIDALGO 20 DE AGOSTO DE 2026



ÍNDICE

1. Introducción y Marco Normativo	2
2. Caracterización del Proceso Inicial (Estado AS-IS)	2
2.1. Descripción del Flujo Empírico Previo	2
2.2. Representación del Proceso Previo (AS-IS)	3
2.3. Identificación de Ineficiencias e Incidentes Críticos	3
3. Diagnóstico y Análisis de Brecha (Gap Analysis)	4
3.1. Objetivos Estratégicos de la Reingeniería	4
4. Diseño e Implementación del Proceso Reingenierizado (Estado TO-BE)	5
4.1. Representación del Proceso Reingenierizado (TO-BE)	5
4.2. Formalización de las 6 Actividades IEEE 14764	5
4.2.1. Actividad 1: Implementación del Proceso (Process Implementation)	5
4.2.2. Actividad 2: Análisis de Problemas y Modificaciones (Problem and Modification Analysis)	5
4.2.3. Actividad 3: Implementación de la Modificación (Modification Implementation)	6
4.2.4. Actividad 4: Revisión y Aceptación del Mantenimiento (Maintenance Review/Acceptance)	6
4.2.5. Actividad 5: Migración (Migration)	6
4.2.6. Actividad 6: Retiro de Software (Software Retirement)	6
5. Integración del Proceso Reingenierizado con SCRUM	6
5.1. Mapeo Estándar IEEE 14764 vs. SCRUM	7
5.2. Estimación de Esfuerzo mediante Planning Poker y Story Points	7
6. Evaluación de Impacto y Métricas Cuantitativas (Resultados)	7
6.1. Definición Matemática de Métricas	8
6.2. Comparativa de Resultados: Estado AS-IS vs. Estado TO-BE	8
7. Conclusiones y Plan de Sostenibilidad	9
7.1. Principales Aprendizajes y Lecciones Aprendidas	9
7.2. Plan de Sostenibilidad para la Fase de Desarrollo de Módulos	9
Referencias Bibliográficas	10



I INTRODUCCIÓN Y MARCO NORMATIVO

El mantenimiento de software representa la fase más extensa y costosa dentro del ciclo de vida del desarrollo de sistemas, absorbiendo históricamente entre el 60 % y el 80 % de los recursos totales del proyecto. De acuerdo con la guía del cuerpo de conocimiento de ingeniería de software **SWEBOK v4.0** y el estándar internacional **ISO/IEC/IEEE 14764:2014**, el mantenimiento no debe concebirse como un esfuerzo reactivo e improvisado de corrección de errores, sino como un proceso disciplinado, repetible, medible y normado orientado a preservar y mejorar la calidad del software a lo largo del tiempo.

El proyecto **Yoliztli** es una plataforma web interactiva diseñada para la enseñanza y preservación de lenguas originarias en niños de educación básica. El sistema está construido bajo una arquitectura monolítica híbrida que combina un backend en **Laravel 10** con un frontend dinámico en **React.js** enlazado mediante **Inertia.js**, respaldado por una base de datos relacional **PostgreSQL**.

En las etapas iniciales de la intervención de mantenimiento, el equipo operaba bajo un esquema informal y empírico. La falta de automatización, la inexistencia de puertas de calidad estáticas y la ausencia de una trazabilidad rigurosa entre fallos detectados y solicitudes de cambio derivaron en incidentes críticos en producción (como la falla de renderizado **PR-001**), la acumulación de deuda técnica y una tasa inicial de fallos del 25 % en las pruebas funcionales de caja negra.

Para solventar estas deficiencias, la presente actividad documenta la aplicación de una **Reingeniería de Procesos de Negocio (BPR — Business Process Reengineering)** al proceso de mantenimiento del sistema Yoliztli. Se toma como marco de referencia estructural el estándar **ISO/IEC/IEEE 14764:2014**, adaptándolo e integrándolo armónicamente con el marco ágil **SCRUM** y herramientas avanzadas de análisis estático y dinámico (**SonarQube**, **PHP-CS-Fixer**, **ESLint** y **Vite**).

2 CARACTERIZACIÓN DEL PROCESO INICIAL (ESTADO AS-IS)

2.1 DESCRIPCIÓN DEL FLUJO EMPÍRICO PREVIO

Antes de la intervención de reingeniería, la atención de mantenimientos dentro del proyecto Yoliztli se realizaba mediante un flujo reactivo e informal, carente de estandarización documental y controles de calidad automatizados. El proceso inicial (**Estado AS-IS**) se estructuraba en las siguientes fases operativas:

1. **Reporte Verbal o Informal:** Los fallos o solicitudes de cambio se notificaban de manera verbal o por chat sin un formato normalizado (ausencia de un protocolo formal de Problem Reports — PR).
2. **Modificación Directa en Código Source:** El desarrollador asignado realizaba cambios de manera directa en las ramas principales (main o develop) sin aislamiento en Sandbox ni pruebas previas de regresión.
3. **Ausencia de Análisis Estático Local:** No existían linters ni formateadores de código configurados en los entornos locales.

4. **Compilación Manual Implícita:** Los assets de React/Inertia no se compilaban de forma automatizada mediante scripts de integración continua, confiando en la ejecución manual del desarrollador.
5. **Despliegue Directo e Inspección Empírica:** Se desplegaba la aplicación y se probaba de manera visual directa sin matriz de casos de prueba formalizados.

2.2 REPRESENTACIÓN DEL PROCESO PREVIO (AS-IS)

En la Figura 1 se ilustra el diagrama de flujo del proceso previo (AS-IS), evidenciando la falta de bucles de control de calidad y la ausencia de validaciones antes del despliegue.

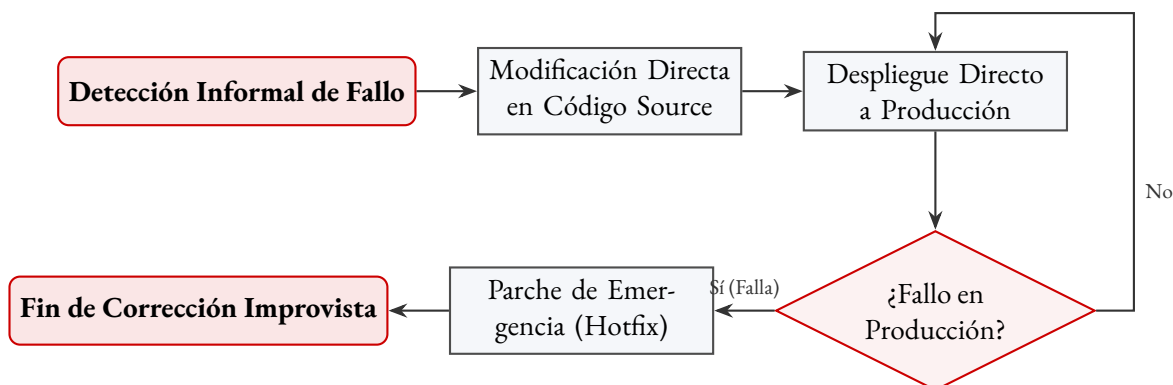


Figura 1. Diagrama de Flujo del Proceso de Mantenimiento Inicial (Estado AS-IS).

2.3 IDENTIFICACIÓN DE INEFICIENCIAS E INCIDENTES CRÍTICOS

El diagnóstico realizado sobre el Estado AS-IS puso de manifiesto fallos severos que comprometían la confiabilidad del sistema Yoliztli:

- **Incidente Crítico PR-001 (Ausencia de Vistas React):** Debido a la falta de un proceso de compilación y verificación de artefactos, se desplegó una versión donde el directorio `resources/js/Pages/` estaba completamente omitido. Esto provocó una excepción 500 fatal e inoperabilidad total de las vistas React enlazadas por Inertia.js.
- **Acumulación de Bugs de Confiabilidad:** El primer diagnóstico automático con **SonarQube v25.2.0** arrojó **67 bugs de confiabilidad** (Clasificación C) y **115 Code Smells**, representando una deuda técnica acumulada de **6 horas y 54 minutos**.
- **Fallas en Pruebas Funcionales (CP007 y CP008):** La primera ejecución de la matriz de pruebas de caja negra registró la falla del caso **CP007** (filtro de cuentos por categoría/edad inoperante) y del caso **CP008** (incapacidad de persistir el progreso del alumno en PostgreSQL), obteniendo una tasa de éxito de apenas el 75 %.
- **Inexistencia de Trazabilidad:** Imposibilidad de auditar qué desarrollador realizó una modificación, bajo qué justificación y qué impacto tuvo sobre otros módulos.

3 DIAGNÓSTICO Y ANÁLISIS DE BRECHA (GAP ANALYSIS)

Para fundamentar la reingeniería, se realizó un análisis de brecha comparando la práctica previa frente a los requisitos establecidos en el estándar **ISO/IEC/IEEE 14764:2014**. En la Tabla 1 se sintetizan los hallazgos principales.

Tabla 1. Matriz de Análisis de Brecha: Estado AS-IS vs. Estándar ISO/IEC/IEEE 14764.

Actividad	IEEE	Práctica Previa (AS-IS)	Brecha e Ineficiencia
14764			
1. Implementación del Proceso		Plan inexistente; sin linters locales ni ambiente Sandbox aislado.	Modificaciones directas en ramas principales; diferencias entre entornos.
2. Análisis de Problemas		Reportes verbales; clasificación nula; ausencia de Problem Reports (PR).	Sin análisis de impacto ni priorización basada en severidad.
3. Implementación del Cambio		Codificación informal sin análisis de mantenibilidad ni estimación de horas.	Generación de deuda técnica (115 smells) y 67 bugs de confiabilidad.
4. Revisión y Aceptación		Verificación visual rápida; sin pruebas de regresión ni auditoría estática.	Fuga de errores a producción (error 500 por PR-001 y fallas CP007/CP008).
5. Migración		Migraciones manuales ocasionales sin respaldos ni pruebas previas.	Riesgo de corrupción o pérdida de datos de progreso de alumnos.
6. Retiro		Omitido; acumulación de código muerto o componentes deprecados.	Incremento innecesario en la complejidad del código (LOC).

3.1 OBJETIVOS ESTRATÉGICOS DE LA REINGENIERÍA

A partir del diagnóstico, se establecieron cuatro objetivos cuantitativos de reingeniería:

- 1. Garantía de Calidad Estática (Quality Gate A):** Reducir los 67 bugs de confiabilidad a cero y resolver más del 90 % de los Code Smells.
- 2. Eficacia en Pruebas de Regresión (EPR = 100 %):** Lograr la remediación total de los casos CP007 y CP008 y asegurar cero fuga de errores a producción.
- 3. Estabilización del MTTR:** Establecer un tiempo medio de resolución (MTTR-MR) inferior a 24 horas por modificación.
- 4. Formalización Normada:** Documentar e implementar rigurosamente las 6 actividades del estándar ISO/IEC/IEEE 14764 integradas con SCRUM.

4 DISEÑO E IMPLEMENTACIÓN DEL PROCESO REINGENIERIZADO (ESTADO TO-BE)

4.1 REPRESENTACIÓN DEL PROCESO REINGENIERIZADO (TO-BE)

El nuevo proceso de mantenimiento (**Estado TO-BE**) transforma el flujo reactivo en una secuencia estructurada con puertas de calidad automatizadas, entornos Sandbox aislados y auditorías obligatorias. La Figura 2 muestra el diagrama BPMN reingenierizado.

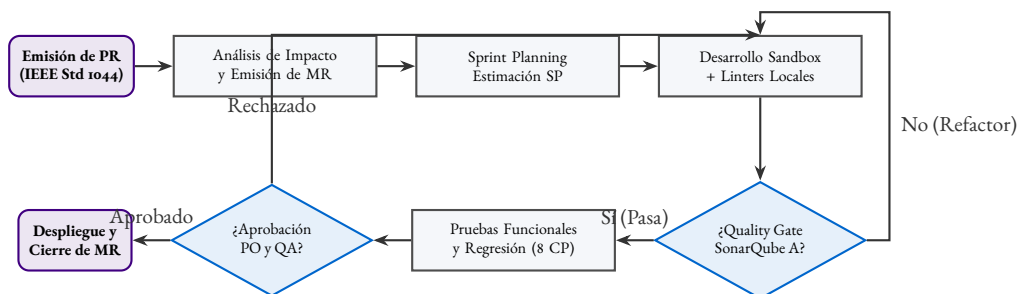


Figura 2. Diagrama de Flujo del Proceso Reingenierizado (Estado TO-BE basado en ISO/IEC/IEEE 14764).

4.2 FORMALIZACIÓN DE LAS 6 ACTIVIDADES IEEE 14764

4.2.1 ACTIVIDAD 1: IMPLEMENTACIÓN DEL PROCESO (PROCESS IMPLEMENTATION)

Se estableció la infraestructura y las reglas operativas del equipo de mantenimiento:

- **Política de Ramas (GitFlow):** Se prohibió la edición directa en la rama `main`. Todo cambio debe desarrollarse en ramas de tipo `feature/MR-XXX` o `fix/MR-XXX`.
- **Herramientas de Linteo Automático:** Configuración local e integración pre-commit de **PHP-CS-Fixer** para el backend Laravel (estándar `PSR-12`) y **ESLint** para el frontend React.
- **Entorno Sandbox Aislado:** Despliegue de servidores locales sincronizados con base de datos de pruebas en PostgreSQL.

4.2.2 ACTIVIDAD 2: ANÁLISIS DE PROBLEMAS Y MODIFICACIONES (PROBLEM AND MODIFICATION ANALYSIS)

- **Problem Reports (PR):** Registro formal de cada anomalía detectada utilizando la clasificación del estándar **IEEE Std 1044-2009** (Campos: ID, Módulo, Descripción, Severidad, Prioridad).
- **Solicitudes de Modificación (MR):** Evaluación de factibilidad técnica y estimación de esfuerzo. Clasificación en 4 categorías:
 - a) *Mantenimiento Correctivo:* Solución inmediata de bugs (ej. MR-001 y MR-002).
 - b) *Mantenimiento Preventivo:* Erradicación de deuda técnica y smells (ej. MR-003).



- c) *Mantenimiento Adaptativo*: Ajuste de lógica de negocio y controladores (ej. MR-004).
- d) *Mantenimiento Perfectivo*: Adición de persistencia de progreso del alumno (ej. MR-005).

4.2.3 ACTIVIDAD 3: IMPLEMENTACIÓN DE LA MODIFICACIÓN (MODIFICATION IMPLEMENTATION)

- Codificación modular respetando la separación de capas (Controladores Eloquent en Laravel, Vistas en React e Inertia Adapter).
- Compilación automatizada de activos frontend utilizando **Vite** (`npm run build`) previniendo incidentes de ausencia de código compilado.

4.2.4 ACTIVIDAD 4: REVISIÓN Y ACEPTACIÓN DEL MANTENIMIENTO (MAINTENANCE REVIEW/ACCEPTANCE)

- **Auditoría Estática con SonarQube v25.2.0**: Evaluación obligatoria del repositorio. Ningún código se acepta en producción si no cumple con la clasificación **Rating A** en Confiabilidad, Seguridad y Mantenibilidad.
- **Ejecución de Pruebas de Caja Negra**: Aplicación de la matriz de 8 casos de prueba funcionales (CP001 a CP008).
- **Firma de Aprobación por Product Owner**: Validación funcional y de UX previa al merge.

4.2.5 ACTIVIDAD 5: MIGRACIÓN (MIGRATION)

- Creación de scripts de migración versionados en Laravel (`database/migrations/`) para gestionar cambios estructurales en la base de datos PostgreSQL sin pérdida de información de usuarios existentes.

4.2.6 ACTIVIDAD 6: RETIRO DE SOFTWARE (SOFTWARE RETIREMENT)

- Protocolo de deprecación formal para eliminar componentes React monolíticos redundantes o rutas obsoletas en `routes/web.php`.

5 INTEGRACIÓN DEL PROCESO REINGENIERIZADO CON SCRUM

Para maximizar la agilidad y el control operacional, las 6 actividades del estándar ISO/IEC/IEEE 14764 se mapearon directamente a los artefactos, roles y eventos del marco **SCRUM**, operando en **Sprints de 2 semanas de duración**.

5.1 MAPEO ESTÁNDAR IEEE 14764 VS. SCRUM

En la Tabla 2 se detalla la correspondencia entre ambos marcos metodológicos.

Tabla 2. Mapeo de Actividades IEEE 14764 a Ceremonias y Artefactos SCRUM.

Actividad IEEE 14764	Evento / Artefacto SCRUM	Responsable Principal
Implementación del Proceso	Sprint Planning / Setup de Entorno	Scrum Master / Dev Team
Análisis de Problemas y MR	Product Backlog Refinement / PR Creation	Product Owner / QA Auditor
Implementación del Cambio	Sprint Execution (Daily Standups)	Development Team
Revisión y Aceptación	Sprint Review & Quality Gate Audit	Product Owner / Scrum Master
Migración y Despliegue	Increment Release / CI Pipeline	Development Team / DevOps
Retiro de Software	Sprint Retrospective (Cleanup)	Scrum Team

5.2 ESTIMACIÓN DE ESFUERZO MEDIANTE PLANNING POKER Y STORY POINTS

En el proceso reingenierizado, el dimensionamiento de las solicitudes de modificación (MR) abandona la estimación intuitiva en horas directas y adopta la técnica ágil de **Planning Poker** basada en la secuencia modificada de Fibonacci (1, 2, 3, 5, 8, 13 Story Points).

Se definió una constante empírica de conversión para el proyecto Yoliztli:

$$1 \text{ Story Point (SP)} \approx 2,0 \text{ horas de desarrollo efectivo} \quad (I)$$

El Backlog total de mantenimiento refinado sumó **20 Story Points (40.90 horas efectivas)** distribuidos en 2 Sprints, según la asignación técnica mostrada en la Tabla 3.

Tabla 3. Desglose de Estimación y Asignación por Solicitud de Modificación (MR).

ID MR	Categoría IEEE 14764	Sprint	Puntos (SP)	Horas Base	Costo Neto (\$)
MR-001	Reconstitución (Correctivo)	Sprint 1	8 SP	16.00 h	\$5,600.00 MXN
MR-002	Remoción Bugs (Correctivo)	Sprint 2	3 SP	5.58 h	\$1,953.00 MXN
MR-003	Deuda Técnica (Preventivo)	Sprint 2	1 SP	1.32 h	\$462.00 MXN
MR-004	Filtros Lógicos (Adaptativo)	Sprint 2	4 SP	8.00 h	\$2,800.00 MXN
MR-005	Persistencia (Perfectivo)	Sprint 1	5 SP	10.00 h	\$3,500.00 MXN
Total	—	Sprints 1 y 2	20 SP	40.90 h	\$14,315.00 MXN

6 EVALUACIÓN DE IMPACTO Y MÉTRICAS CUANTITATIVAS (RESULTADOS)

Para validar la efectividad de la reingeniería aplicada, se monitorearon **cuatro métricas fundamentales de calidad y proceso** basadas en ISO/IEC/IEEE 14764 y SWEBOK v4.0.

6.1 DEFINICIÓN MATEMÁTICA DE MÉTRICAS

1. **Tiempo Medio de Resolución por Modificación (MTTR-MR):** Mide la eficiencia temporal del equipo desde la recepción de una MR hasta su despliegue verificado:

$$MTTR-MR = \frac{\sum_{i=1}^N (T_{\text{resolución},i} - T_{\text{aprobación},i})}{N} = \frac{88,9 \text{ h}}{5 \text{ MR}} = 17,78 \text{ horas/MR} \quad (2)$$

2. **Eficacia de Pruebas de Regresión (EPR):** Evalúa la capacidad del proceso para impedir la fuga de errores a producción:

$$EPR = \frac{B_{\text{detectados}}}{B_{\text{detectados}} + B_{\text{fuga}}} \times 100 = \frac{8}{8 + 0} \times 100 = 100 \% \quad (3)$$

3. **Porcentaje de Remediación de Deuda Técnica (PRDT):** Cuantifica el grado de saneamiento del código fuente respecto al diagnóstico inicial de SonarQube:

$$PRDT = \frac{D_{\text{removida}}}{D_{\text{inicial}}} \times 100 = \frac{110 \text{ Code Smells}}{115 \text{ Code Smells}} \times 100 = 95,65 \% \quad (4)$$

4. **Índice de Mantenibilidad de Software (IM):** Métrica ponderada estandarizada basada en el Volumen de Halstead (V), Complejidad Ciclomática (G) y Líneas de Código (LOC):

$$IM = 171 - 5,2 \ln(V) - 0,23(G) - 16,2 \ln(LOC) = 44,82 \text{ pts (Clasificación Aceptable/Moderada)} \quad (5)$$

6.2 COMPARATIVA DE RESULTADOS: ESTADO AS-IS VS. ESTADO TO-BE

En la Tabla 4 se demuestra el impacto directo de la reingeniería de procesos sobre la calidad del proyecto Yoliztli.

Tabla 4. Comparativa Cuantitativa de Indicadores de Calidad (AS-IS vs. TO-BE).

Métrica / Indicador	Proceso Previo (AS-IS)	Proceso Reingenierizado (TO-BE)	Mejora Obtenida
Bugs de Confiabilidad	67 bugs (Rating C)	0 bugs (Rating A)	100 % Erradicados
Code Smells (Deuda)	115 smells (6h 54m)	5 smells (Deuda < 15m)	95.65 % Remediado
Tasa de Éxito Pruebas	75.0 % (6/8 aprobados)	100.0 % (8/8 aprobados)	+25.0 % Incremental
Fuga de Errores a Prod.	Incidente Crítico PR-001 (Error 500)	0 incidentes en producción	Estabilidad Total
Linteo Automático	Inexistente (manual)	Cobertura 100 % (PHP-CS/ESLint)	Estandarizado
Trazabilidad PR → MR	0 % (comunicación verbal)	100 % trazable en Git/Backlog	Formalizado



7 CONCLUSIONES Y PLAN DE SOSTENIBILIDAD

La reingeniería del proceso de mantenimiento aplicada al proyecto **Yoliztli** demostró ser una decisión estratégica de alto impacto. La transición desde un modelo empírico e informal hacia un proceso formalizado bajo el estándar **ISO/IEC/IEEE 14764:2014** e integrado con **SCRUM** permitió erradicar de forma sistemática la totalidad de los bugs de confiabilidad, recuperar la arquitectura monolítica híbrida (Laravel + React) y estabilizar la persistencia del progreso infantil en PostgreSQL.

7.1 PRINCIPALES APRENDIZAJES Y LECCIONES APRENDIDAS

1. **La automatización previene el error humano:** La incorporación de linters locales y compilación automatizada con Vite eliminó de raíz la posibilidad de repetir incidentes críticos como la ausencia de activos (PR-001).
2. **Las puertas de calidad reducen costos:** El filtrado estático previo mediante SonarQube evitó que errores sintácticos o de tipado llegaran a fases avanzadas de prueba, reduciendo el costo de corrección.
3. **La estimación ágil mejora la predictibilidad:** La adopción de Planning Poker y Story Points permitió al equipo cumplir el plan de mantenimientos dentro del margen presupuestario (\$14,665.00 MXN consolidados con costos indirectos) sin sobrepasos temporales.

7.2 PLAN DE SOSTENIBILIDAD PARA LA FASE DE DESARROLLO DE MÓDULOS

Con la estructura de mantenimiento formalizada y normada, el equipo se encuentra preparado para iniciar la siguiente fase del proyecto: la programación, despliegue y pruebas de integración del resto de los módulos funcionales del sistema Yoliztli (como la ampliación de catálogos de cuentos y nuevos minijuegos lúdicos). Se mantendrá el cumplimiento estricto del estándar IEEE 14764 y la supervisión continua en SonarQube para evitar la inyección de nueva deuda técnica en producción.



REFERENCIAS BIBLIOGRÁFICAS

- [1] **ISO/IEC/IEEE 14764:2014.** *Software Engineering — Software Life Cycle Processes — Maintenance*. International Organization for Standardization / IEEE Computer Society.
- [2] **IEEE Computer Society** (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*, Version 4.0. IEEE Computer Society Press.
- [3] **IEEE Std 1044-2009.** *IEEE Standard Classification for Software Anomalies*. IEEE Computer Society.
- [4] **Pressman, R. S., & Maxim, B. R.** (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- [5] **Schwaber, K., & Sutherland, J.** (2020). *La Guía de Scrum: Las Reglas del Juego*. Scrum.org.